

How Far Can a Feed-Forward Map Take a Robot?

Anonymous Authors

Abstract—An image-navigation system can keep a graph of images, objects or pixels, or reconstruct once and plan in 3D. Feed-forward reconstruction makes the latter tempting: a video produces posed geometry in one pass. If that geometry is ready for navigation, the relative maps and learned controllers used by topological methods become unnecessary. We test this premise. We rasterise the predicted floor, join adjacent cells under a step-height rule and execute routes with perfect localisation on ObjectReact’s HM3D split and a Unitree Go2. The floor graphs often connect and the local geometry looks plausible, yet planned routes miss the true navigable floor because the predicted trajectory places surfaces in the wrong physical coordinates. Floor IoU and route-local error can detect this misplacement. For Scal3R, reusing predicted depth at odometry poses turns many failed maps into drivable ones while connectivity stays unchanged. A camera-height check provides a scene-reference-free consistency test before the robot moves. Relative image and topological methods remain useful because each motion is conditioned on the current view. Current feed-forward maps are not yet reliable for direct metric planning. Training losses should penalise geometric misplacement along routes, and evaluation should measure the same failure.

I. INTRODUCTION

A robot is driven through a scene once. It must later retrace the route, return to a place it saw or avoid an obstacle using that recording. We call this first recording the mapping run.

The mapping run can be kept in two forms. Relative methods connect images, objects or pixels in a graph and plan between them [1], [2], [3], [4], [5], [6]. Metric methods reconstruct the scene and use a geometric planner. That planner assumes that every surface occupies the correct global coordinate.

Recent feed-forward models turn a sequence of images into geometry and camera poses in one pass [7], [8], [9], [10], [11], [12]. Some also predict metric scale. We ask whether a robot can plan and drive directly on that output.

We first test whether the reconstructed floor connects the mapping run’s endpoints. We then plan on that floor and execute the route with perfect localisation. A route can pass through the correct doors in the predicted map yet miss those doors in the room (Fig. 1). Connectivity describes the predicted map, but it does not say where that map lies in the physical scene.

We compare the predicted floor and planned route with Habitat’s navigable surface. Floor overlap and route displacement predict arrival better than connectivity. The test is whether a path in the map occupies the same floor as the path the robot must drive.

Depth and pose leave these models together. To separate them, we place the model’s depth at recorded poses. We run this test in HM3D and drive the resulting maps on a Unitree Go2.

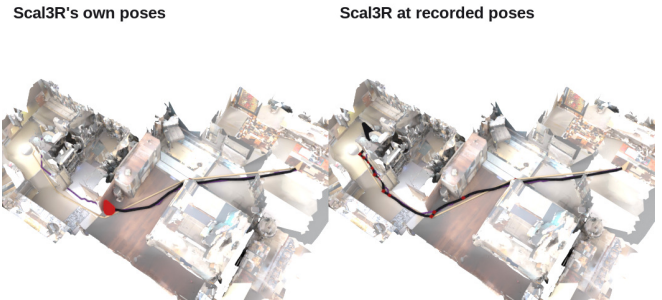


Fig. 1: Scal3R’s per-frame depth on the same episode under two trajectories. With Scal3R’s poses (left), the planned route hits a wall. At the recorded poses (right), it reaches the goal. Plum is the predicted route, tan is the simulator route, black is the driven path, and red marks collisions and the failed stop.

Contributions.

- We test whether feed-forward 3D maps can replace relative navigation maps by planning and executing on outputs from four frontends under perfect localisation.
- We show that map connectivity misses navigation-critical placement errors by comparing predicted floors and planned routes with Habitat’s navmesh.
- We isolate pose error by rebuilding Scal3R maps from the same predicted depth at recorded odometry poses, improving closed-loop execution in simulation and on a Unitree Go2.
- We quantify how camera-height consistency, field of view, frame spacing and robot width affect whether the predicted floor supplies a route.

II. RELATED WORK

Navigation from relative observations. The established form of this idea pairs a topological graph of previously seen images with a learned relative controller [13], [14], [15]. RoboHop connects image segments and drives relative to the selected segment [1]. ObjectReact replaces its image-plane costs with relative 3D object distances and learns control from the resulting WayObject map [3]. TANGO takes the other branch: it keeps RoboHop’s topological global plan and builds a local depth map while the robot moves [2]. MAST3R-Nav carries ObjectReact’s design from objects to matched pixels, and PixelLoop adds edges between pixels seen again in nonadjacent frames [4], [5]. LoTIS follows a separate route. It predicts where the recorded trajectory lies in the current image and steers from that prediction [6]. Each method anchors motion to the current observation. We test

whether one fused feed-forward map can instead supply the floor before the robot moves.

Metric navigation on feed-forward reconstruction. Planning on reconstructed geometry can account for traversability in principle. DUST3R [7] and VGGT [8] regress pointmaps in one pass. Scal3R [9], VGGT-Omega [10] and LoGeR [11] extend this idea to long sequences, while AMB3R [12] and MapAnything [16] target metric output. MapAnything can also condition its reconstruction on camera poses. None evaluates navigation over its own reconstruction. Prior depth and odometry work suggests that indoor reconstruction depends on short frame baselines and a wide lens [17], [18]. We test frame spacing and a cropped-input condition.

Metric mapping. Classical metric systems estimate and revise the camera trajectory as they build a map. TaskSLAM evaluates them through repeated navigation because trajectory error alone does not measure route repeatability [19]. Our recorded-pose Scal3R experiment uses the platform trajectory while Scal3R supplies every depth map.

Offline metrics against closed-loop behaviour. Planning KL divergence [20] scores a detector through the trajectory planned on its output. Schreier et al. [21] compare detector metrics with closed-loop driving, and dos Santos et al. [22] show that the BEV map with the highest IoU need not drive best. We pair every map score with its rollout on the same episode.

Occupancy metrics. Occupancy prediction usually scores a voxel set by intersection over union [23], [24], [25]. clDice [26], Boundary IoU [27] and RayIoU [28] address topology, boundary placement and ray consistency. MapEx [29] and PixelLoop [5] also report planning-aware scores. Two maps with the same number of wrong cells can drive differently when those cells fall along different routes.

III. SETUP

Walks. We use the 33 episodes in ObjectReact’s [3] HM3D [30] instance-image-goal evaluation split [31]. Each mapping walk follows the simulator’s shortest path from the episode start to its goal. The camera records 320×240 RGB with a 120° horizontal field from 1.31 m above the floor. A walk contains 26 to 131 frames and covers a median navmesh distance of 8.96 m.

Frontends. We compare Scal3R [9], VGGT-Omega [10], LoGeR [11] and MapAnything [16]. For the RGB-only comparison in Table I, each map stays in the model’s output frame, anchored at its first camera. The true frame-0 pose places the completed map for evaluation, and ground truth scores where the robot stops. The pooled 132-map analysis uses the first three frontends and Scal3R + PGO. Scal3R+PGO keeps the predicted depth and refines only the poses after inference. We run MapAnything with its official settings, with and without input poses. Section VII instead replaces predicted poses with odometry.

IV. METHOD

We turn each reconstruction into a floor graph and measure how much of it the goal can reach (Fig. 2).

Scale. A reconstruction has an unknown scale, but its predicted camera track identifies the cells that the camera traversed. Let $\mathcal{C}$ be the cells under that track, widened by a half-width $w = 0.20$ m. For such a cell u , let $z(u)$ be the predicted floor height and $z_{\text{cam}}(u)$ the camera height as it passed above, both in reconstruction units. We compare their median difference with the measured camera height h .

$$\hat{h} = \text{median}_{u \in \mathcal{C}} (z_{\text{cam}}(u) - z(u)), \quad \gamma = \frac{\hat{h}}{h}. \quad (1)$$

Equation (1) uses model outputs and the known camera height h . A metric model gives $\gamma = 1$. A threshold of ℓ metres becomes $\gamma\ell$ map units, and a path of length $\hat{\ell}$ map units becomes $\hat{\ell}/\gamma$ metres.

Floor. We keep every second frame and every fourth pixel in each direction. Each retained pixel gives one 3D point. We fuse them into one cloud and rasterise it on the grid. The gauge converts its cell size to metres: a 5 cm cell is 0.05γ reconstruction units. Let $\mathcal{M}$ be the mapped cells. The first card in Fig. 2 shows both sampling steps.

We choose these strides to keep the cloud small. Keeping every frame and pixel gives Scal3R 32 times the points, yet all 33 maps stay connected. It maps more floor, while the reachable share falls from 85% to 75% because dense sampling adds cells outside the goal’s component. Every comparison therefore uses the same sampling.

A cell is *certified* when it lies in $\mathcal{C}$, and *observed* when its 3×3 neighbourhood $N(u)$ is flat enough to stand on,

$$\max_{v \in N(u)} z(v) - \min_{v \in N(u)} z(v) < \gamma \sigma_{\text{flat}}, \quad (2)$$

with $\sigma_{\text{flat}} = 0.12$ m, the value ETH’s traversability filter ships for ANYmal [32]. The floor $\mathcal{F} \subseteq \mathcal{M}$ is every cell that is certified or observed, so a cell the robot has driven over is floor whether or not it passes (2).

Two adjacent floor cells are crossable when their height difference is below the robot’s step limit. Two 8-adjacent cells $u, v \in \mathcal{F}$ carry an edge when

$$|z(u) - z(v)| \leq \tau, \quad \tau = \max(\gamma \sigma_{\text{step}}, \nu), \quad (3)$$

with $\sigma_{\text{step}} = 0.20$ m. The first term is Habitat’s navmesh test. Its builder, Recast, joins two neighbouring cells when the step between them is at most the robot’s maximum climb, which Habitat sets to 0.20 m [33], [34]. Our map and the navmesh therefore use the same limit. The second term ν handles noisy maps. On the cells the camera traversed, the floor should lie $\hat{h}$ below the camera. We set ν to the 90th percentile of its deviation from that height. We compute one ν per walk and leave it uncapped. It can only add edges, so it makes connectivity easier to achieve.

An edge costs its physical length with a penalty for observed floor: $w(u, v) = s(u, v)(p(u) + p(v))/2$, where $s(u, v)$ is the distance between the cells, and p is 1 on certified floor and 1.2 on observed floor. The 20% penalty is just the preference for floor traversed during mapping.

Scoring a map. A Dijkstra search from the goal cell over these edges gives every cell a cost $d(u)$. A map *connects*

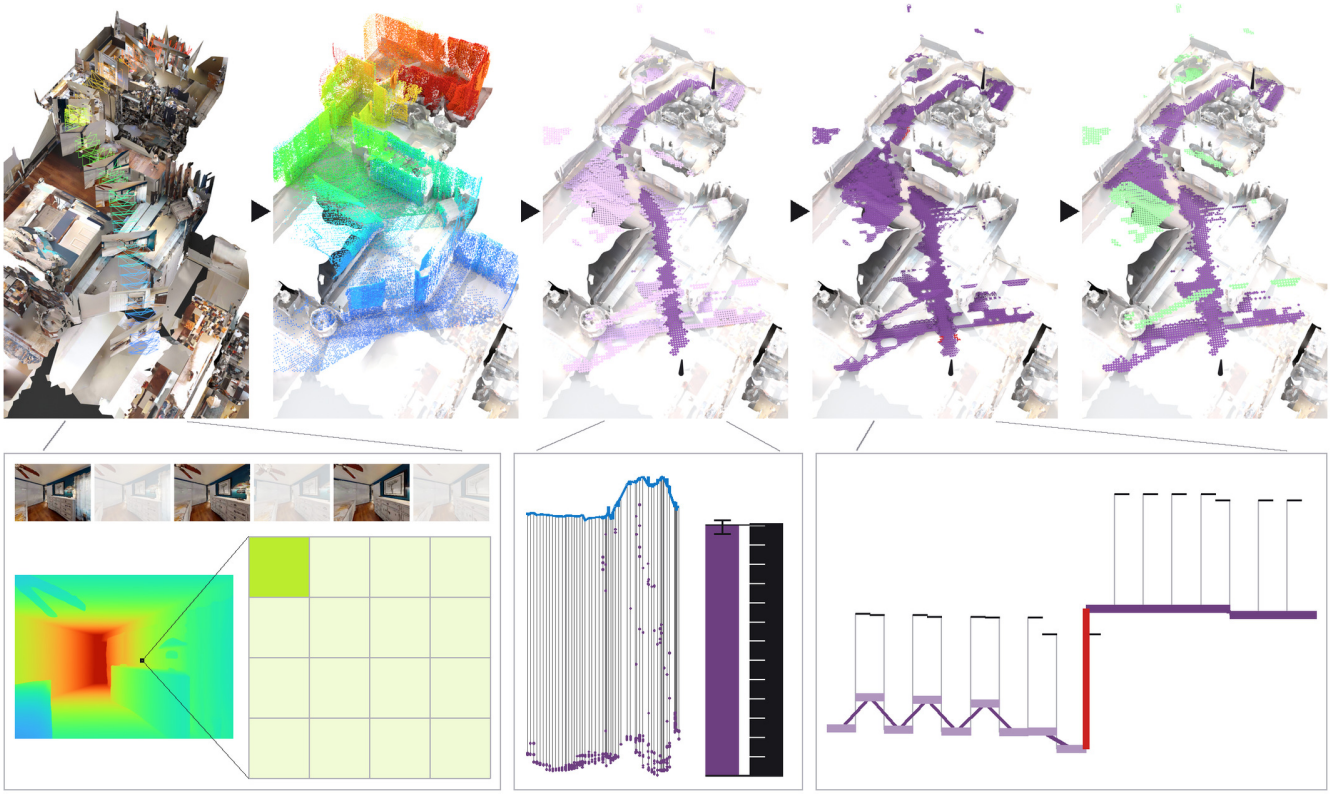


Fig. 2: The method on one episode, from frames to the floor the goal reaches. Plum is reached, green is mapped but unreachable.

when some cell of the 21×21 window W around the first camera is reached at all, $\min_{u \in W} d(u) < \infty$. If the goal cell itself is unmapped, the search starts from the nearest mapped cell, with no cap on that distance. *Reachable floor* is the share of the map's own floor that the search covers,

$$R = \frac{|\{u \in \mathcal{F} : d(u) < \infty\}|}{|\mathcal{F}|}. \quad (4)$$

The half-width w changes which cells enter $\mathcal{F}$. Equation (3) then tests connectivity for a point robot.

Posed Scal3R. A robot with odometry knows where each frame was taken. So we build one more map that keeps the model's depth and throws away its poses. Frame i goes at the pose T_i the platform recorded in place of the pose $\hat{T}_i$ the model inferred. That leaves the scale. We fit a similarity from the model's camera centres $\hat{c}_i$ to the recorded ones c_i ,

$$(s^*, R^*, t^*) = \arg \min_{s > 0, R \in SO(3), t \in \mathbb{R}^3} \sum_i \|sR\hat{c}_i + t - c_i\|^2, \quad (5)$$

and multiply the predicted depth by s^* . R^* and t^* are thrown away, since the recorded poses already place every frame in metres. The resulting map uses the recorded trajectory and the track-derived depth scale. We measure the gauge afterwards as a consistency check. The recorded metre frame sets every threshold and rollout waypoint.

TABLE I: Four RGB-only frontends on the 33 walks.

frontend	connects	reachable floor median	mean	gauge median	cells median
Scal3R	33 / 33	85.0%	82.2%	0.992	5463
VGGT-Omega	31 / 33	85.4%	78.4%	0.979	4386
LoGeR	32 / 33	82.0%	80.2%	0.317	6069
MapAnything	2 / 33	7.2%	19.8%	1.016	7194

V. CONNECTIVITY RESULTS

Scal3R, VGGT-Omega and LoGeR connect nearly every walk (Table I). LoGeR connects 32 of 33 walks despite a gauge of 0.317. MapAnything has a gauge of 1.016 but connects only 2 of 33. Agreement with the rig height does not produce a coherent floor graph on its own.

A. Field of view

The gauge of (1) needs predicted floor along the camera track. On a straight, level walk, a camera at height h with vertical field ϕ sees no floor closer than

$$d_{\text{blind}} = h / \tan(\phi/2), \quad (6)$$

so a walk shorter than d_{blind} cannot observe its own future path. Turns and revisits can create overlap sooner. At $h = 1.31$ m, this distance is 2.33 m for a 58.8° vertical field and 1.01 m for the full 104.8° field (Fig. 3, top).

We crop each 320×240 frame to its middle 82×104 pixels (width by height), a $47.9^\circ \times 58.8^\circ$ view that matches

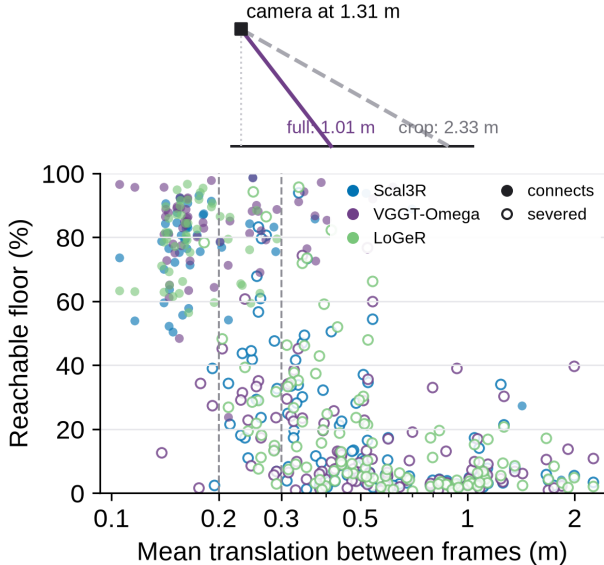


Fig. 3: Input conditions for the floor graph. Top, the nearest floor seen by the full and cropped cameras. Bottom, the frame-spacing sweeps for all three RGB frontends. Colour identifies the frontend. Filled maps connect and open maps are severed.

the RGB-D camera in Robot@Home2 [35], and leave the rest of the pipeline unchanged. Scal3R then loses 10 connections and a median 13.4 points of reachable floor. The crop also lowers angular resolution. The experiment measures both changes together.

The practical requirement is that the camera see floor close enough to overlap the path travelled during mapping. The full input begins seeing floor at 1.01 m and connects every walk, whereas the crop begins at 2.33 m and connects 23. Camera height and vertical field of view should therefore be reported together: either can change how much near-field floor is available to the reconstruction.

B. Frame spacing

We run Scal3R, VGGT-Omega and LoGeR on uniformly spaced subsets of each walk, always keeping the first and last frames (Fig. 3, bottom).

With all frames, every map in Fig. 4 connects. When frames are removed, the models still predict floor near the camera track, but gaps split it into separate pieces. More frames do not always help a single walk. We summarise each input by its mean camera spacing, $\bar{s} = L/(n-1)$, where L is the path length and n is the number of frames. Across all walks, reachable area drops as spacing grows, with ρ from -0.663 to -0.842 . Each frontend connects at least 35 of 39 maps below 0.20 m per frame. Above 0.30 m, only 14 of 289 maps connect. The full HM3D sequences already exceed five frames per metre, so the full-input experiment does not expose this failure.

TABLE II: Routes and arrivals over 33 walks. The 0.10 m gate matches the executor’s collision radius. MapAnything was not rerun with this gate. Habitat’s oracle navmesh already uses the executor radius.

route source	point graph		0.10 m gate	
	route	arrive	route	arrive
Scal3R	33 / 33	2 / 33	30 / 33	1 / 33
VGGT-Omega	31 / 33	0 / 33	28 / 33	1 / 33
LoGeR	32 / 33	6 / 33	27 / 33	3 / 33
Scal3R + PGO	30 / 33	0 / 33	22 / 33	0 / 33
common pool	126 / 132	8 / 132	107 / 132	5 / 132
MapAnything	2 / 33	1 / 33	–	–
Scal3R, recorded poses	32 / 32	26 / 32	30 / 32	28 / 32
Oracle navmesh	–	–	32 / 33	32 / 33

VI. CLOSED-LOOP RESULTS

A. Rollout and controls

We plan from the predicted first camera to the predicted last over the graph of (3), then drive the plan in Habitat-sim 0.2.4 [34]. The agent has a 0.1 m collision radius. The true frame-0 pose and the gauge place the completed map in the simulator for evaluation. The RGB-only maps and plans use no ground-truth pose after frame 0. Section VII reports separate controls that use recorded poses. The controller reads its pose from the simulator. We use perfect localisation, so failures point to the predicted route.

BLIND follows the graph’s shortest path with pure pursuit at a 0.30 m lookahead. Habitat provides 10° turns and 0.25 m forward steps. The action budget is $100 + 4L/0.25$ for a plan L metres long, rounded to an integer. A watchdog stops the rollout when it advances less than 0.50 m over 40 actions.

Completing the waypoint list and succeeding are separate outcomes. We follow ObjectReact’s 1.0 m success threshold [3]. Reaching the last predicted waypoint does not count if that waypoint is in the wrong place. Across the 132 common map trials, tightening the radius to 0.5 m gives 2 arrivals and relaxing it to 2.0 m gives 17. Either oracle gives 32 at every radius.

The oracle tests whether the same controller can drive a valid route. It plans between the true endpoints on Habitat’s navmesh, then uses BLIND to follow that route. We run the route once at Habitat’s native corner spacing and once resampled at the grid planner’s waypoint spacing. Both versions arrive on 32 of 33 walks. On the remaining walk, the two endpoints lie on different navmesh islands and no oracle route exists.

We test Scal3R, VGGT-Omega and LoGeR on each of the 33 walks, plus Scal3R after its pose-graph optimisation. Four map variants on 33 walks give $4 \times 33 = 132$ trials under the common protocol. Although their point graphs find 126 routes, BLIND arrives only 8 times (Table II). Matching the graph to the executor leaves 107 routes and 5 arrivals. MapAnything finds two routes and one arrives. Scoring the pooled 132 point-graph trials against each map’s predicted endpoint raises the total from 8 to 9. Endpoint error accounts for one failure, while the same controller reaches 32 of 33 true endpoints on either oracle route.

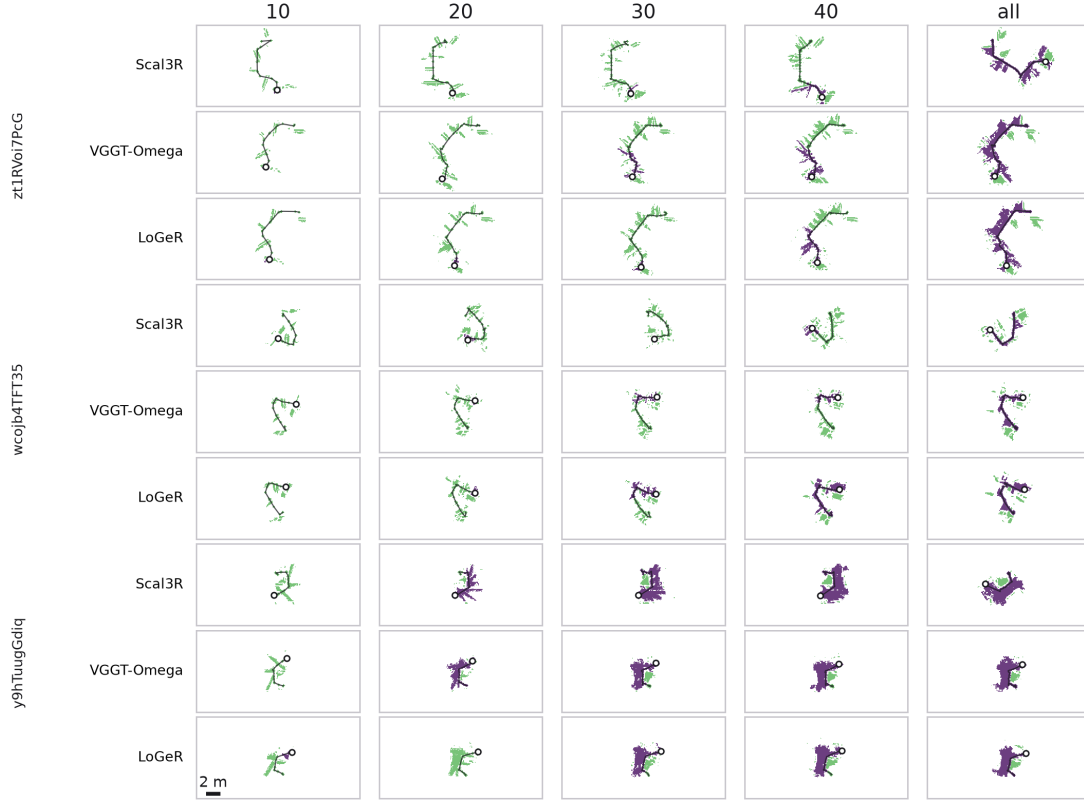


Fig. 4: Three walks reconstructed from five input budgets. Deep plum is mapped floor reachable from the goal (ring), green is mapped floor outside that component, and black is the camera track. Every cell covers the same 23×11.5 m area. Each row is centred in its frontend’s output frame. The rows show the 20th-percentile, median and maximum retained-floor cases among the 13 episodes scored by all three frontends.

A shortest path can hug an obstacle boundary, so CLEAR-ANCE changes the price of the same graph edges to favour open floor. Across the same 132 trials, it raises arrivals from 8 to 16. It adds no cells and cannot repair floor placed in the wrong location.

When sampled every 0.25 m, a median 12–20% of the route lies more than 0.20 m from Habitat’s navmesh, depending on the frontend. Of the pooled 132 point-graph trials, 117 stop through the watchdog after repeated collisions. The farther a route stays on the navmesh, the farther the robot drives ($r = 0.59$). These rollouts record a median 49 collisions before they stop.

B. Predicting arrival from the map

Connectivity is necessary because graph following fails on every severed map. It cannot tell us which connected map will work. Across the 132 BLIND trials, reachable floor and arrival have $r = -0.05$, with an episode-bootstrap interval of $[-0.19, 0.08]$. The correlation stays near zero with clearance-aware planning ($r = -0.04$) and when the goal is already within 1 m ($r = -0.02$). Table III compares the 126 maps that yield a route.

Map-only scores do not separate arrivals from failures. Reachable floor and camera-height error have AUCs of 0.34

TABLE III: Association with arrival on the same 126 planned maps. The arrow gives the arrival direction. Intervals resample episodes. AP chance is 0.063. “True floor” is Habitat’s navmesh after the map is placed as in its rollout.

score	true floor	AUC [95% interval]	AP
reachable floor $\uparrow$	no	.34 [.12, .61]	.06
camera-height error $\downarrow$	no	.26 [.08, .44]	.05
walk length $\downarrow$	no	.77 [.54, .93]	.21
free-floor IoU $\uparrow$	yes	.70 [.53, .84]	.12
Boundary IoU $\uparrow$	yes	.80 [.65, .91]	.21
missing-floor p90 $\downarrow$	yes	.85 [.70, .95]	.27
route off-floor share $\downarrow$	yes	.90 [.75, .99]	.46

and 0.26 (Table III). They never compare where the predicted floor lies in the physical scene. Walk length alone reaches 0.77, above free-floor IoU at 0.70.

We place each prediction exactly as in its rollout using frame 0, the first-camera heading and γ . We make no later pose fit. On a fixed 5 cm grid within 1.6 m of the recorded walk, Boundary IoU reaches AUC 0.80 and missing-floor distance reaches 0.85. The share of the planned route outside true floor reaches AUC 0.90 and average precision 0.46. The corridor crop does not count as a floor boundary. Only eight maps arrive, so the bootstrap intervals cannot separate the

route score from missing-floor distance. These diagnostics require Habitat’s true floor, which a deployed robot does not have.

MapAnything is not part of the pooled AUC. Its camera-height gauge is near one, but its median free-floor IoU is 0.15 and its missing-floor distance is 1.60 m. The pooled maps score 0.38 and 0.46 m. A map can therefore have the right vertical scale and still place little floor where the robot needs it. MapAnything has one arrival, too few for an association score on that arm alone.

C. Robot width

The graph in Table I treats the robot as a point. The executor is a 0.20 m diameter disc, so we plan again after removing edges within 0.10 m of unknown floor. Equation (3) still handles height changes, and the start and goal are snapped after this width check. The results are in Table II. With the robot’s width included, recorded-pose Scal3R arrives 28 times and original Scal3R once. The width check can select a different route, so a stricter graph can produce more arrivals. Doubling the half-width to 0.20 m leaves only 5 of 132 maps connected. The planner converts width through γ , so a wrong gauge also gives it the wrong robot width.

D. Parameters

The flatness threshold barely changes execution. Sweeping it from 0.06 to 0.30 m changes at most one arrival. We also make the adaptive noise term permissive because a severed graph cannot produce a route. Across the three raw frontends and three refinement variants, it connects 186 of 198 maps. Removing it leaves 161. Capping it at the 0.50 m furniture height leaves 184 and changes reachable floor by 0.5 points. We therefore use the uncapped term in the connectivity comparisons.

VII. POSED MAPS

By a feed-forward map we mean geometry built from the mapping video alone, with the robot’s recorded trajectory kept out of the reconstruction. Here we use the robot’s recorded poses. Scal3R still supplies depth, but we place each depth map at its recorded pose and fit one global depth scale from the two camera tracks. HM3D poses are exact, so this result is an upper bound for pose replacement.

All 32 paired floor graphs connect with either trajectory. BLIND arrives on 2 with Scal3R’s trajectory and 26 with the recorded one (Fig. 1). This adds 24 paired arrivals. The recorded-pose maps and their waypoints stay in metres. The camera-height gauge is measured but never applied to them.

We next perturb each incremental recorded pose with synthetic yaw and translation noise and compose the errors along the walk, using three seeds at each nonzero level (Fig. 6). The zero-noise control gives 26 arrivals. The mean falls to 20.7 at 1.0% median endpoint drift, 15.7 at 2.2%, 8.7 at 4.3% and 4.7 at 9.8%. Median reachable floor stays between 82.5 and 84.1%. Pose error therefore erodes the exact-pose gain before the connectivity score notices. The

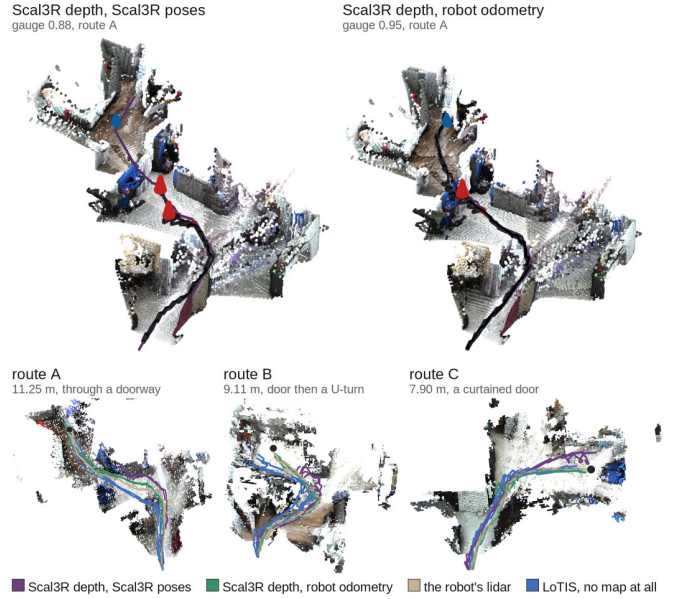


Fig. 5: Top, route A from the same Scal3R depth at two sets of poses: plum the plan, black every driven run, red a collision, a pin where a run did not finish, blue the walk’s end. Bottom, the three routes from above on the robot’s lidar, with all 34 runs, one colour per case.

experiment uses accumulated synthetic noise and does not evaluate VIO.

MapAnything can take camera poses as input, so we also run it with the exact simulator poses. Both runs use every frame and the official 518×392 preprocessing and output mask. MapAnything still predicts its own trajectory. Pose input raises connections from 2 to 16 and median reachable floor from 7% to 73%, but arrivals fall from one to zero. The gauge falls from 1.016 to 0.601. This test measures pose conditioning inside MapAnything. Posed Scal3R is different: it discards the predicted poses and places depth directly at the recorded poses.

Both Scal3R pose variants connect all 32 maps, but replacing its poses raises arrivals from 2 to 26. The camera-height error $|\gamma - 1|$ separates the 26 arrivals from the six failures at AUC 0.73. Large values flag a scale inconsistency before driving. Route execution still depends on where the poses place the floor.

A. Driving a quadruped

We drive the maps on a Unitree Go2. Its front camera sits 0.37 m above the floor, and we undistort its fisheye to $80^\circ \times 51^\circ$. We teleoperate three routes through one home and cut each into a walk of 70 to 104 frames. Route A is 11.25 m through a doorway, route B is 9.11 m with a near U-turn just past its door, and route C is 7.90 m through a curtained one. Every door is 0.85 m wide or narrower, and the robot stands still for every frame.

Each walk gives one Scal3R run and four cases. Two maps carry the same Scal3R depth, placed once at the model poses and once at the robot’s odometry. The third uses the robot’s

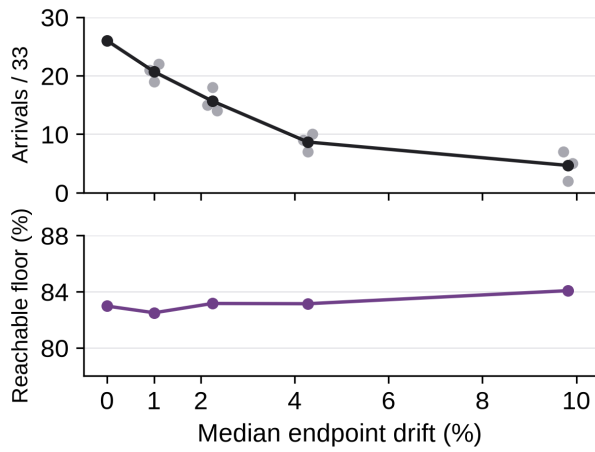


Fig. 6: Recorded-pose Scal3R under accumulated synthetic pose noise. Grey points are the three noise seeds. Arrivals fall as endpoint drift grows, while median reachable floor stays nearly flat.

TABLE IV: The Go2 runs. The first two maps are the same Scal3R depth at two sets of poses, the third is the robot’s own lidar, and LoTIS carries no map. Goal is the map’s endpoint error before the robot moves. Endpoint is the median driven error in the robot’s odometry. Both are in metres.

route	map	gauge	goal	finished	endpoint
A	Scal3R, own poses	0.88	1.09	0/2	3.11
	Scal3R, odometry	0.95	0.45	2/3	0.37
	lidar	metric	0.00	2/2	0.19
	LoTIS	–	–	2/3	0.67
B	Scal3R, own poses	0.87	0.66	0/3	1.64
	Scal3R, odometry	1.01	0.02	3/3	0.23
	lidar	metric	0.01	2/3	0.18
	LoTIS	–	–	2/3	0.78
C	Scal3R, own poses	0.90	0.67	2/3	0.62
	Scal3R, odometry	1.02	0.13	3/3	0.30
	lidar	metric	0.01	3/3	0.22
	LoTIS	–	–	3/3	0.51

lidar at the odometry poses and checks that a route through the rooms exists. LoTIS [6] carries no metric map and locates the recorded frames in the live image while driving, so we report it as a separate navigation alternative. Both Scal3R maps take one depth scale from the odometry track and are laid out by their measured gauge. The lidar map is metric. The three metric maps for each route all connect and reach 96.4–100% of their own floor. Connectivity and reachable floor do not distinguish them before execution. Across the three walks, Scal3R’s endpoint error is 11.9–14.8% of the walk’s span.

BLIND drives each map’s plan from a taped start, on the point-robot graph used in simulation, three trials per case where the session allowed (Table IV, Fig. 5). The model poses finish 2 of 8 runs (Fig. 7), the odometry poses 8 of 9, the lidar 7 of 8 and LoTIS 7 of 9. On route C, model poses give endpoint errors of 0.514–0.652 m, compared with 0.202–0.352 m for odometry poses using the same

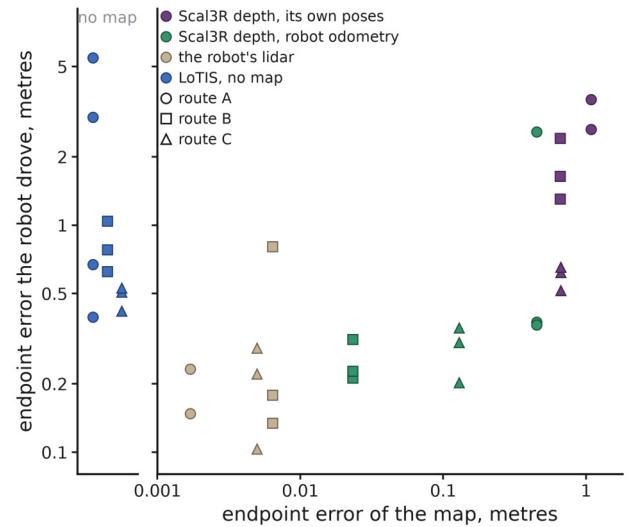


Fig. 7: Every Go2 run. Across is the map’s endpoint error before the robot moves. Up is how far from the walk’s end the robot stopped. Both axes are logarithmic. LoTIS sits in the strip because it carries no map.

depth. With model poses, the median driven error is 0.618 m and the map endpoint error is 0.667 m. Its $\gamma = 0.904$ scale lengthens the planned route by 10.6%. The lidar map places the endpoint within 0.005 m, yet the robot stops 0.221 m away. This gap measures the combined follower and odometry error in the odometry frame. It does not establish the robot’s physical endpoint error.

On route C, a spotter moved the robot towards the goal in 8 of 12 runs. All endpoint errors use robot odometry because we did not measure them with a tape. With three trials per case, the smallest attainable two-sided p -value is 0.10.

VIII. CONCLUSION

The tested frontends often reconstruct plausible local surfaces and connect the mapping run’s endpoints. The route still fails when pose error puts those surfaces at the wrong coordinates. Connectivity cannot detect that shift. A score against the true floor can detect it, but a robot does not have the true floor at deployment.

Scal3R shows how much of the failure comes from pose. Replacing its poses with exact recorded poses raises arrivals from 2 of 32 to 26 of 32, even though every graph connects under both pose sets. Synthetic drift removes this gain while reachable floor stays almost constant. The camera-height check catches gross scale errors before driving, but it does not certify a route.

Feed-forward reconstruction cannot yet replace relative navigation by being rasterised and planned on once. It may reconstruct the right surfaces at the wrong coordinates. Relative methods remain useful because each action is tied to the current view. Direct metric mapping needs route-placement losses and benchmarks that execute the resulting routes.

ACKNOWLEDGMENTS

We used ChatGPT and Claude for paraphrasing and writing assistance during manuscript preparation. All generated content was reviewed and verified by the authors.

REFERENCES

- [1] S. Garg, *et al.*, “RoboHop: Segment-based topological map representation for open-world visual navigation,” in *ICRA*, 2024, pp. 4090–4097.
- [2] S. Podgorski, S. Garg, M. Hosseinzadeh, L. Mares, F. Dayoub, and I. Reid, “TANGO: Traversability-aware navigation with local metric control for topological goals,” in *ICRA*, 2025, pp. 2399–2406.
- [3] S. Garg, *et al.*, “ObjectReact: Learning object-relative control for visual navigation,” in *CoRL*, 2025.
- [4] V. Garg, *et al.*, “MASt3R-Nav: WayPixel navigation in relative 3D maps,” in *ICRA*, 2026.
- [5] S. Chittawar, *et al.*, “PixelLoop: Shortcut topological navigation with pixel-level loops,” in *IROS*, 2026.
- [6] F. L. Busch, *et al.*, “LoTIS: Learning to localize reference trajectories in image-space for visual navigation,” in *RSS*, 2026.
- [7] S. Wang, V. Leroy, Y. Cabon, B. Chidlovskii, and J. Revaud, “DUST3R: Geometric 3D vision made easy,” in *CVPR*, 2024.
- [8] J. Wang, M. Chen, N. Karaev, A. Vedaldi, C. Rupprecht, and D. Novotny, “VGGT: Visual geometry grounded transformer,” in *CVPR*, 2025.
- [9] T. Xie, *et al.*, “Scal3R: Scalable test-time training for large-scale 3D reconstruction,” in *CVPR*, 2026.
- [10] J. Wang, *et al.*, “VGGT- Ω ,” in *CVPR*, 2026.
- [11] J. Zhang, *et al.*, “LoGeR: Long-context geometric reconstruction with hybrid memory,” arXiv:2603.03269, 2026.
- [12] H. Wang and L. Agapito, “AMB3R: Accurate feed-forward metric-scale 3D reconstruction with backend,” in *CVPR*, 2026.
- [13] D. Shah, A. Sridhar, A. Bhorkar, N. Hirose, and S. Levine, “Gnm: A general navigation model to drive any robot,” in *2023 IEEE International Conference on Robotics and Automation (ICRA)*, 2023, pp. 7226–7233.
- [14] D. Shah, *et al.*, “ViNT: A foundation model for visual navigation,” in *CoRL*, 2023.
- [15] A. Sridhar, D. Shah, C. Glossop, and S. Levine, “Nomad: Goal masked diffusion policies for navigation and exploration,” in *2024 IEEE International Conference on Robotics and Automation (ICRA)*, 2024, pp. 63–70.
- [16] N. Keetha, *et al.*, “MapAnything: Universal feed-forward metric 3D reconstruction,” in *3DV*, 2026.
- [17] A. Duzceker, S. Galliani, C. Vogel, P. Speciale, M. Dusmanu, and M. Pollefeys, “DeepVideoMVS: Multi-view stereo on video with recurrent spatio-temporal fusion,” in *CVPR*, 2021.
- [18] Z. Zhang, H. Rebecq, C. Forster, and D. Scaramuzza, “Benefit of large field-of-view cameras for visual odometry,” in *ICRA*, 2016.
- [19] Y. Du, S. Feng, C. G. Cort, and P. A. Vela, “Task-driven SLAM benchmarking for robot navigation,” in *IROS*, 2025.
- [20] J. Philion, A. Kar, and S. Fidler, “Learning to evaluate perception models using planner-centric metrics,” in *CVPR*, 2020.
- [21] T. Schreier, K. Renz, A. Geiger, and K. Chitta, “On offline evaluation of 3D object detection for autonomous driving,” in *ICCV Workshops*, 2023.
- [22] F. C. dos Santos, E. Antonelo, and G. C. K. Couto, “Closed-loop evaluation of bird’s-eye-view maps from cross-view transformers as inputs to behavior-cloning policies,” arXiv:2609.05783, 2026.
- [23] X. Tian, *et al.*, “Occ3D: A large-scale 3D occupancy prediction benchmark for autonomous driving,” in *NeurIPS*, 2023.
- [24] H. Yu, Y. Wang, Y. Chen, and Z. Zhang, “Monocular occupancy prediction for scalable indoor scenes,” in *ECCV*, 2024.
- [25] C. Zhou, Y. Luo, and C. Chen, “Generalizing visual geometry priors to sparse Gaussian occupancy prediction,” in *CVPR*, 2026.
- [26] S. Shit, *et al.*, “clDice - a novel topology-preserving loss function for tubular structure segmentation,” in *CVPR*, 2021.
- [27] B. Cheng, R. Girshick, P. Dollar, A. C. Berg, and A. Kirillov, “Boundary IoU: Improving object-centric image segmentation evaluation,” in *CVPR*, 2021.
- [28] H. Liu, *et al.*, “Fully sparse 3D occupancy prediction,” in *ECCV*, 2024.
- [29] C. Ho, *et al.*, “MapEx: Indoor structure exploration with probabilistic information gain from global map predictions,” in *ICRA*, 2025.
- [30] S. K. Ramakrishnan, *et al.*, “Habitat-matterport 3D dataset (HM3D): 1000 large-scale 3D environments for embodied AI,” in *NeurIPS Datasets and Benchmarks*, 2021.
- [31] J. Krantz, S. Lee, J. Malik, D. Batra, and D. S. Chaplot, “Instance-specific image goal navigation: Training embodied agents to find object instances,” arXiv:2211.15876, 2022.
- [32] M. Wermelinger, P. Fankhauser, R. Diethelm, P. Krusi, R. Siegwart, and M. Hutter, “Navigation planning for legged robots in challenging terrain,” in *IROS*, 2016, pp. 1184–1189.
- [33] Mikko Mononen and contributors, “Recast navigation,” <https://github.com/recastnavigation/recastnavigation>, 2024.
- [34] M. Savva, *et al.*, “Habitat: A platform for embodied AI research,” in *ICCV*, 2019.
- [35] G. Ambrosio-Cestero, J.-R. Ruiz-Sarmiento, and J. Gonzalez-Jimenez, “The Robot@Home2 dataset: A new release with improved usability tools,” *SoftwareX*, vol. 23, p. 101490, 2023.